



Moroccan School of Engineering Sciences of Tangier

END OF MODULE PROJECT

Specialty

Computer Engineering and Networks (3IIR)

Project Title

**Design and Development of a Multimedia
Library Management System (MediaForge)**

Realized by:

Nizar EL IDRYSY

Hajar CHABLI

Nizar BTIRA

Academic Supervisor

Pr. Youssef JDIDOU

Jury Members

Pr. Youssef JDIDOU

Submission Date: January 7, 2026

Dedication

To our dearest professor, Mr. Youssef JDIDOU,

For his invaluable kindness, enlightened guidance, and exceptional pedagogy that transformed our learning into a true passion. This project is a modest testimony of our gratitude for your constant dedication to our academic and professional success.

To our parents,

Sources of our motivation and unwavering support, who never ceased to believe in us. No words can express our gratitude for your sacrifices.

To all our friends and colleagues of the 3IIR class, with whom we share this engineering adventure.

Acknowledgements

Before beginning this report, we would like to extend our deepest thanks to all those who contributed, directly or indirectly, to the realization of this project.

First and foremost, we warmly thank the administration of the **Moroccan School of Engineering Sciences (EMSI)** for providing us with a stimulating learning environment conducive to creativity and excellence.

We express our profound gratitude to our supervisor, **Pr. Youssef JDIDOU**, who guided us with patience and rigor. His wise advice on software architecture and C++ best practices was decisive in the success of this project. His lectures and practical sessions formed the solid foundation upon which we built this application.

Finally, we thank all those who participated in the testing phases of the application, allowing us to identify and correct imperfections to deliver a quality finished product.

Contents

Dedication	1
Acknowledgements	2
List of Figures	6
1 General Introduction	7
1.1 Project Context	7
1.2 Problem Statement	7
1.3 Project Objectives	8
1.4 Report Structure	8
2 Analysis and Requirements Specification	9
2.1 Actor Identification	9
2.1.1 The Administrator	9
2.1.2 The Client (Member)	9
2.2 Functional Requirements Specification	10
2.2.1 Global Use Case	10
2.3 Technical and Environmental Constraints	10
3 Architecture and Design	11
3.1 Object-Oriented Paradigm	11
3.1.1 Inheritance	11
3.1.2 Polymorphism	11

3.2	Data Structures	12
3.2.1	Memory Management with Smart Pointers	12
3.2.2	File System	12
4	Implementation and Development	13
4.1	The UI Class: A Textual Graphics Engine	13
4.1.1	ANSI Color Management	13
4.1.2	Loading Animation	13
4.2	Core Business Logic	14
4.2.1	The User Class	14
4.2.2	Search Engine	14
4.3	Error Handling and Robustness	15
4.3.1	Securing Numeric Inputs	15
4.3.2	Missing File Handling	15
5	Results Presentation	16
5.1	Home Interface	16
5.2	Client The dashboard	17
5.3	Test Scenario: Purchasing Media	17
5.4	Test Scenario: Administration	18
6	General Conclusion and Outlook	19
6.1	Project Assessment	19
6.2	Current Limitations	19
6.3	Future Improvements	20

Bibliography and Webography	21
------------------------------------	-----------

List of Figures

3.1	Media Class Hierarchy	11
5.1	System Login Interface	16
5.2	Client Main Menu with Navigation Options	17

Chapter 1

General Introduction

1.1 Project Context

The automation of management processes has become an inescapable necessity for any modern organization. In the educational and cultural sector, multimedia libraries play a central role as vectors of knowledge and entertainment. However, manual management of a large number of resources (books, DVDs, audio files, audiobooks) and users (students, professors, staff) quickly becomes a tedious task prone to human error.

It is in this context that our project **MediaForge System** falls. It is a technological response to the challenges faced by school multimedia library managers.

1.2 Problem Statement

How to design a computer system capable of efficiently managing the cycle of borrowing, the diversity of multimedia supports, and different user access levels, while remaining simple to use for non-expert users?

The main challenges are:

- Diversity of media: A book does not have the same properties as a video (duration vs number of pages).
- Conflict management: A media item cannot be borrowed by two people simultaneously.
- Data security: User information must be protected.
- User experience: A console application can be austere; the challenge is to make it attractive.

1.3 Project Objectives

The goal of this project is to develop a complete application in C++ language that allows to:

1. **Centralize data:** Group all media and users in a single system.
2. **Automate tasks:** Availability calculation, basket management, search filtering.
3. **Secure access:** Implementation of a robust authentication system.
4. **Offer a friendly interface:** Use of colors and structured menus to guide the user.

1.4 Report Structure

This document is structured to logically present the project progress:

- **Chapter 2** analyzes the needs and specifies the features.
- **Chapter 3** details the technical architecture and UML design.
- **Chapter 4** dives into code implementation and error handling.
- **Chapter 5** presents the final results through usage scenarios.

Chapter 2

Analysis and Requirements Specification

2.1 Actor Identification

System analysis allowed identifying two main actors interacting with the application.

2.1.1 The Administrator

The administrator is the library manager. They possess the highest privileges. Their role is to keep the system operational and up to date. Their responsibilities include:

- Creation of new user accounts (students or other administrators).
- Deletion of obsolete accounts.
- Catalog supervision (adding/removing media - simulated by file reading).
- Consultation of global statistics (number of loans, media popularity).

2.1.2 The Client (Member)

The client represents the standard end user, usually a student or a teacher. They use the system to:

- Search for books or multimedia files.
- Build a borrowing basket.
- Validate loans or make purchases.
- Consult their personal history.

2.2 Functional Requirements Specification

2.2.1 Global Use Case

Although this report does not contain a formal graphical UML diagram, we can textually describe the major use cases:

Case N°1: Authentication

Actor: All. **Description:** The user must enter a valid email and password. **Alternative flow:** If credentials are incorrect, the system denies access and emits an audible alert.

Case N°2: Basket Management

Actor: Client. **Description:** The user navigates the catalog, selects an item, and adds it to their temporary basket. They can then validate the basket to transform the content into real loans.

Case N°3: Media Search

Actor: All. **Description:** The user enters a string. The system scans the database and displays all media whose title contains the entered string (case insensitive).

2.3 Technical and Environmental Constraints

To ensure portability and performance, the following constraints were respected:

- **Language:** C++ (Standard 17) for its fine memory management and execution speed.
- **Interface:** Console (CLI) compatible with Unix (macOS/Linux) and Windows terminals.
- **Storage:** Text files (.txt) for maximum portability without dependence on a complex database server (like SQL Server or Oracle).

Chapter 3

Architecture and Design

3.1 Object-Oriented Paradigm

The choice of Object-Oriented Programming (OOP) is strategic. It allows modeling the real world intuitively. A library is composed of objects (Books, Users) that interact with each other.

3.1.1 Inheritance

Inheritance allowed us to factorize code. Instead of rewriting `title` and `price` attributes for each media type, we defined them in a parent class `Media`.

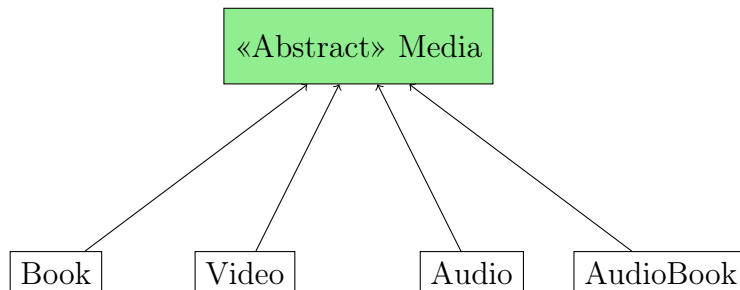


Figure 3.1: Media Class Hierarchy

As illustrated above, `AudioBook` is a hybrid class (representing the flexibility of our polymorphic design).

3.1.2 Polymorphism

Polymorphism is used extensively. The `displayCompact()` method is virtual in the `Media` class and implemented differently in each subclass. This allows displaying a heterogeneous list of media (Books and Videos mixed) by calling the same method, with the system dynamically determining which version to execute.

3.2 Data Structures

3.2.1 Memory Management with Smart Pointers

In modern C++ (C++11 and later), manual memory management with `new` and `delete` is discouraged as it is a source of errors (memory leaks). We opted for `std::shared_ptr`.

- **Advantages:** Automatic memory release when the object is no longer used.
- **Application:** Library vectors contain `shared_ptr<Media>`.

3.2.2 File System

The persistence architecture relies on three pillars:

1. **users.txt:** Account database.
2. **owned.txt:** Association table between users and borrowed media.
3. **baskets.txt:** Temporary backup of baskets in progress.

This approach allows restarting the application without losing the state of transactions.

Chapter 4

Implementation and Development

This chapter presents the most relevant technical aspects of the source code. The full project contains over 1200 lines of code; we will focus on key mechanisms.

4.1 The UI Class: A Textual Graphics Engine

To offer a user experience superior to a simple black and white console, we developed a static UI class.

4.1.1 ANSI Color Management

ANSI escape codes allow controlling text and background color in the terminal.

```
1 #define ansiReset "\033[0m\033[40m"
2 #define bold "\033[1m"
3 #define highlightBlue "\033[44m"
4 #define highlightGreen "\033[42m"
```

Listing 4.1: ANSI Color Macros Definitions

4.1.2 Loading Animation

A `slowPrint` function was created to simulate text being typed character by character, giving a "hacker" or "system processing" effect.

```
1 static void slowPrint(string text, int speedMs) {
2     for (size_t i = 0; i < text.length(); ++i) {
3         cout << text[i] << flush;
4         this_thread::sleep_for(chrono::milliseconds(speedMs));
5     }
6 }
```

Listing 4.2: Typewriter Effect Function

4.2 Core Business Logic

4.2.1 The User Class

This class encapsulates all personal data and possible actions of a user. Note the use of `vector` to store `Media` objects.

```
1 class User {
2 protected:
3     string email;
4     string password;
5     string role;
6     vector<shared_ptr<Media>> basket;
7     vector<shared_ptr<Media>> ownedMedia;
8
9 public:
10    void addToBasket(shared_ptr<Media> m) {
11        if(m->getIsBorrowed()) {
12            cout << "Item is currently borrowed." << endl;
13            return;
14        }
15        basket.push_back(m);
16        cout << "Added to basket." << endl;
17    }
18 };
```

Listing 4.3: User Class Extract

4.2.2 Search Engine

The search functionality is crucial for ergonomics. It is implemented with linear complexity $O(N)$.

```
1 void searchMedia(const vector<shared_ptr<Media>>& library) {
2     string query;
3     UI::inputLine(query);
4     string lowerQuery = toLower(query);
5
6     for(const auto& m : library) {
7         // Case insensitive substring search
8         if(toLower(m->getTitle()).find(lowerQuery) != string::npos) {
9             results.push_back(m);
10        }
11    }
12    // Display results...
```

13 }

Listing 4.4: Search Algorithm

4.3 Error Handling and Robustness

An aspect often overlooked in student projects, but central to ours, is Error Handling. An application must never crash due to unexpected user input.

4.3.1 Securing Numeric Inputs

The classic ‘cin » int’ problem is that if the user enters a letter, the input stream blocks and the program enters an infinite loop. We solved this with the `inputInt` method:

```
1 static void inputInt(int& var) {  
2     string line;  
3     getline(cin, line); // Read the whole line as text  
4     try {  
5         if(line.empty()) { var = -1; return; }  
6         var = stoi(line); // Explicit String to Int conversion  
7     } catch (...) {  
8         var = -1; // Return invalid code on error  
9     }  
10 }
```

Listing 4.5: Secure Integer Input

Thanks to this method, if the user types "abc" instead of "123", the program continues normally displaying an "Invalid choice" message.

4.3.2 Missing File Handling

At startup, the application checks for the existence of data files. If they are missing, they are automatically created (`scanDirectory` method and `System` constructor). This avoids "File Not Found Exception" type errors.

Chapter 5

Results Presentation

This section presents the final rendering of the application as it appears to the user.

5.1 Home Interface

The home screen was designed to be visually striking. It uses a complex ASCII frame representing a server or central terminal.



Figure 5.1: System Login Interface

As shown in Figure 5.1, the user is immediately immersed in the application universe. Input fields appear sequentially.

5.2 Client The dashboard

Once authenticated successfully (the system plays a "success.mp3" sound), the client accesses their dashboard.

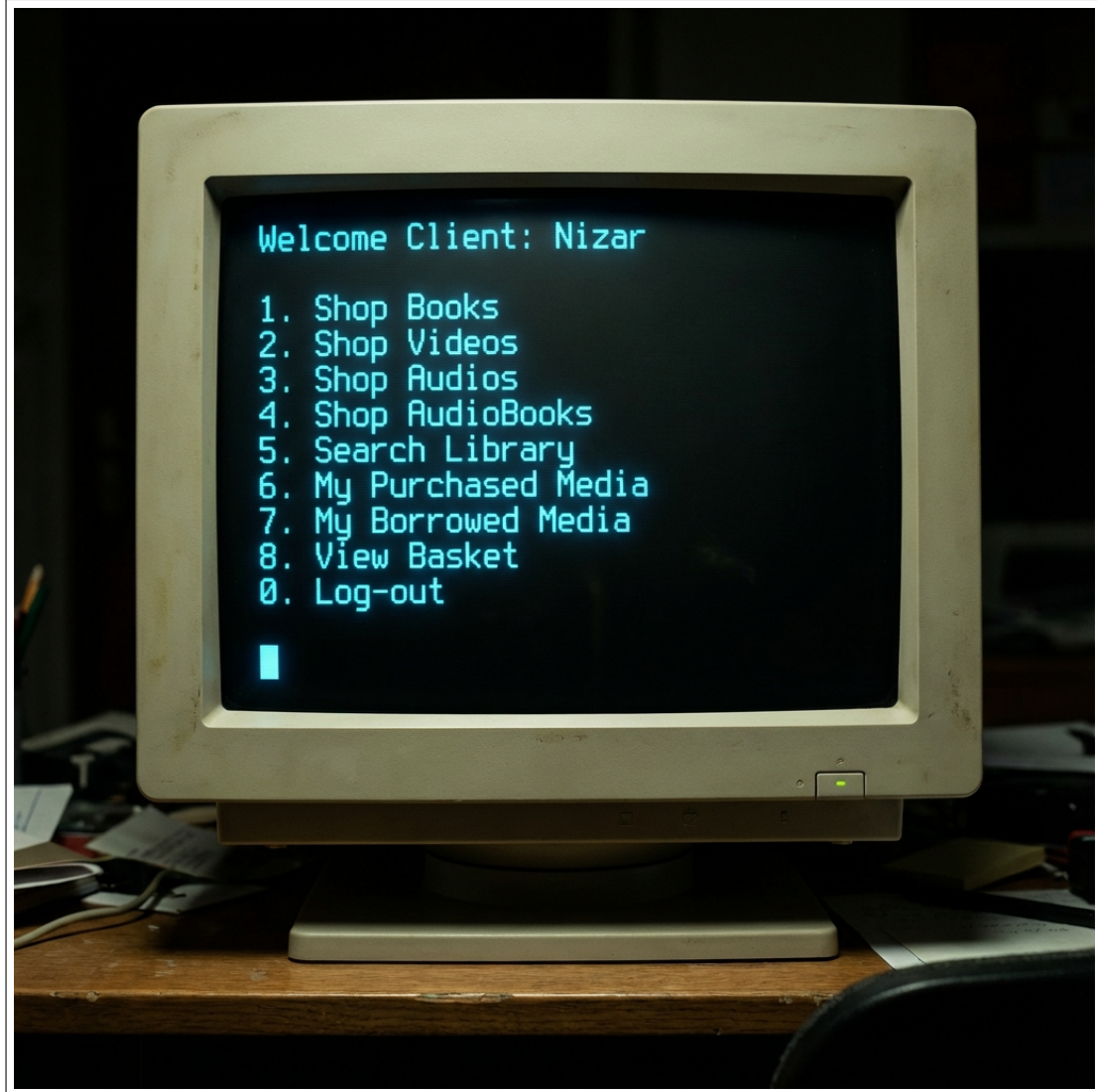


Figure 5.2: Client Main Menu with Navigation Options

The menu (Figure 5.2) clearly displays all available actions. The use of neon blue on specific backgrounds reduces eye strain and recalls professional development interfaces.

5.3 Test Scenario: Purchasing Media

Here is the typical flow of a purchase session:

1. The client selects "1. Shop Books".

2. The list of available books is displayed. Borrowed books are marked in **Red**, available ones in **Green**.
3. The client enters the book number. It is added to the basket.
4. The client goes to "8. View Basket". They see their order total.
5. They select "Confirm Loan" or "Buy". If "Buy" is chosen, the media is marked as purchased and added to their "My Purchased Media" list.

5.4 Test Scenario: Administration

The administrator has a different menu. They can:

- View the list of all registered users.
- Delete a problematic user.
- Export a global report in `report.txt` format containing the stock status at a given time.

Chapter 6

General Conclusion and Outlook

6.1 Project Assessment

At the end of this project, we successfully designed and developed a complete multimedia library management application, named **MediaForge System**.

Technically, this project allowed us to consolidate our skills in:

- **C++ Language:** Mastery of syntax, pointers, references, and STL.
- **Object Design:** Concrete application of abstraction, encapsulation, and polymorphism concepts.
- **Algorithmics:** Implementation of efficient sorting and searching algorithms.
- **Project Management:** Teamwork and source code management.

The initial objective, which was to provide a tool automating loan management while remaining accessible, is fully achieved. The user interface, although in console mode, offers surprising ergonomics thanks to the care given to design.

6.2 Current Limitations

Despite its qualities, the system has certain limitations inherent to its nature as an academic project:

- The absence of a Graphical User Interface (GUI) may deter some non-tech-savvy users.
- Text file storage is not suitable for very large scale (millions of users) due to performance and security reasons.
- The application is single-user (no real client-server mode via network).

6.3 Future Improvements

To evolve this project towards a professional solution, several avenues are conceivable:

1. **Migration to an SQL Database:** Use SQLite or MySQL to secure and accelerate data access.
2. **GUI Development:** Use the Qt framework to create an interface with mouse, buttons, and real book cover images.
3. **Web Version:** Transform the C++ core into a backend (with a framework like Drogon) and create a Web frontend for remote accessibility.
4. **Networking:** Implement a client-server architecture using sockets to allow multiple users to connect simultaneously from different machines.

In conclusion, **MediaForge System** constitutes a solid and scalable foundation, demonstrating the power of C++ to create performant management applications.

Bibliography and Webography

Books

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.
2. Scott Meyers, *Effective Modern C++*, O'Reilly Media, 2014.

Websites Consulted

- **cppreference.com**: Official C++ standard library documentation.
- **stackoverflow.com**: For specific problem solving related to clang compilation errors.
- **overleaf.com**: For LaTeX report writing and compilation.